repositories, the label concept is an abstraction over Git branches, tags, or commit IDs.

**Listing 4.12 Configuring integration between config server and config repo**

```
server:
  port: 8888                                          ❶
  tomcat:
    connection-timeout: 2s
    keep-alive-timeout: 15s
    threads:
      max: 50
      min-spare: 5

spring:
  application:
    name: config-service                              ❷
  cloud:
    config:
      server:
        git:
          uri: <your-config-repo-github-url>          ❸
          default-label: main                         ❹
```

❶ The port that the Config Service application will listen on

❷ A name for the current application

❸ The URL to the remote Git repository to use as the configuration data backend. For example, https://github.com/ PolarBookshop/config-repo.

❹ By default, the server will return configuration data from the “main” branch.

**WARNING** The configuration I used for Config Service assumes the config repo is publicly available on GitHub. When you use private repositories (which is often true for real-world applications), you need to specify how to authenticate with the code repository provider by using additional configuration properties. For more information, refer to the official Spring Cloud Config documentation (https://spring.io/projects/spring-cloud-config). I’ll further discuss handling credentials in chapter 14.

## *4.3.3 Making the configuration server resilient*